

EE-2003

Computer Organization and Assembly Language

Week 08 – Lecture 15 - 16

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Beyond 8-bit and 16-bit Operations
- 3 Concept of Extended Shifting
- 4 Example — 8-bit Extended Addition
- 5 Example — 8-bit Multiplication
- 6 Extended Addition Algorithm
- 7 Extended Subtraction Algorithm
- 8 Example 4.2 — 16-bit Multiplication Code

Quick Review: What was in the Last Lecture?

- Shift and Rotate Instructions
- Comparison of Shift and Rotate Instructions
- Example (4-bit Multiplication)

4-bit Multiplication Review

- Previously, we implemented a 4-bit multiplication algorithm.
- This helped us understand the basic process of multiplication in assembly.
- However, the true power of computers lies in handling **large numbers**.
- We now extend this idea to **16-bit** and **32-bit** multiplications.

Beyond 8-bit and 16-bit Operations

- 8-bit numbers can be multiplied using word registers.
- But what about **32-bit or larger numbers**?
- Assembly provides only the **basic building blocks**.
- Using logic and creativity, we can extend these algorithms indefinitely.
- The limitation is not in assembly language—**it's in imagination**.

16-bit Multiplication Producing a 32-bit Result

- To multiply 16-bit numbers and get a 32-bit result:
 - We must handle **32-bit shifts** and **32-bit additions**.
 - 16-bit shifting alone will cause **data loss**.
- Shifting a 16-bit number left may drop valuable bits from the left.
- “Valuable bits” are those that are **1s**; dropping them changes the result.

Using 32-bit Space for 16-bit Numbers

- Place the 16-bit number in a **32-bit register space**.
- Upper 16 bits are zeroed out.
- This ensures no valuable bits are lost during shifting.
- Even though operands are 16-bit, **32-bit operations** are required for correct results.

Example: Multiplying 40000 (0x9C40)

- Example: 40000 (decimal) = 0x9C40 (hex) = 1001110001000000 (binary)
- Shifting left (multiplying by 2) gives:
 - Result = 0011100010000000
 - Leftmost bit is dropped into the carry flag.
- Expected result = 0x13880 (80000 decimal)
- Actual result = 0x3880 (14464 decimal) → **Incorrect due to bit loss!**

Key Takeaways

- Always be cautious when using **shift operations**.
- Use extended registers (e.g., 32-bit) to prevent bit loss.
- Understand that assembly provides **building blocks**, not limitations.
- Larger arithmetic operations can be achieved through logical extension.

Concept of Extended Shifting

- Our CPU can only shift **word-sized (16-bit)** data at once.
- For larger numbers (like 32-bit), we use an **extended shifting algorithm**.
- It uses only two instructions:
 - SHL word [num1], 1 — shifts the lower word left.
 - RCL word [num1+2], 1 — rotates the upper word through carry.

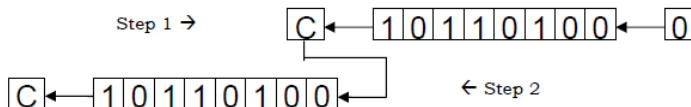
Memory Setup for Extended Shift

- Declaring: `num1: dd 40000`
- DD reserves 32 bits (double word) in memory.
- The value fits in 16 bits, so upper 16 bits are initially zero.
- Lower 16 bits: accessible at `num1`
- Upper 16 bits: accessible at `num1 + 2`

How Extended Shift Works (Left Shift Example)

- Step 1: Shift the lower 16 bits left.
 - Leftmost bit goes into **carry flag**.
- Step 2: Rotate the upper 16 bits through the carry.
 - The bit from carry enters as the new LSB.
 - Both halves now behave as one continuous 32-bit value.

Illustration of Extended Shift



- The right block = lower 16 bits.
- The left block = upper 16 bits.
- Bit from lower half moves to carry, then to upper half.

Right Shift Operation

- For shifting right, the order is reversed:
 - SHR word [num1+2], 1 – shift upper half right.
 - RCR word [num1], 1 – rotate carry into lower half.
- The least significant bit of upper half moves into carry.
- Carry bit enters the lower half from the left side.

Signed vs Unsigned Shifting

- For unsigned numbers, use SHR (logical shift right).
- For signed numbers, use SAR (arithmetic shift right).
- Logical shift fills zeros; arithmetic keeps the sign bit.

Extending Beyond 32 Bits

- The same logic applies for larger numbers (e.g., 64, 128, or even 1024 bits).
- Repeat the rotate-through-carry instruction for each additional word.
- Thus, we can shift any size of number using simple 16-bit operations.
- Limitation: only the **available memory**, not architecture.

Example — 8-bit Extended Addition

Assembly Code

```
[org 0x0100]
jmp start

num1:    dw 0x40FF      ; 4400, 40FF
dest:    dw 0x40FF
src:     dw 0x1001

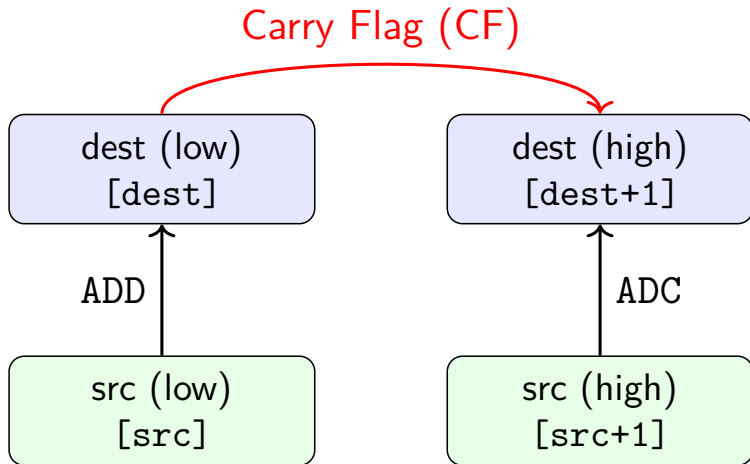
; start:
; shift
shl byte [num1], 1
rcl byte [num1 + 1], 1

; addition
xor ax, ax      ; clear AX
mov al, byte[src]
add byte[dest], al

mov al, [src + 1]
adc byte[dest + 1], al

mov ax, 0x4c00
```

Memory Visualization: 8-bit Extended Addition



Explanation: Low byte is added first using ADD, while the high byte uses ADC to include carry.

Example — 8-bit Multiplication (Part 1)

Assembly Code

```
[org 0x0100]
jmp start

multiplicand:    dw 0x00C8      ; 200 = 0b11001000
multiplier:      db 0x32        ; 50  = 0b00110010
result:          dw 0           ; expected: 10,000 = 0x2710

start:
    mov     cl, 8
    mov     dl, [multiplier]

checkbit:
    shr     dl, 1
    jnc     skip
        mov     al, [multiplicand]
        add     byte [result], al
        mov     al, [multiplicand + 1]
        adc     byte [result + 1], al
```

Example — 8-bit Multiplication (Part 2)

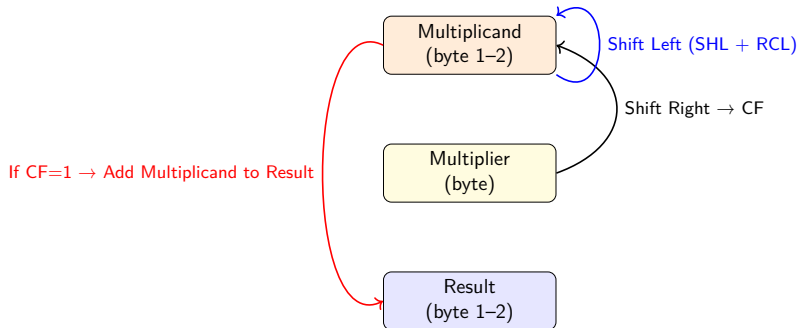
Assembly Code (continued)

```
skip:
    shl     byte [multiplicand], 1
    rcl     byte [multiplicand + 1], 1

    dec     cl
    jnz     checkbit

mov     ax, 0x4c00
int     0x21
```

Memory Layout: 8-bit Multiplication Process



Explanation: Each bit of the multiplier is checked (via CF). If carry = 1, multiplicand is added to result; then multiplicand is shifted left.

Concept of Extension

- For 16-bit multiplication, we need 32-bit addition.
- Assembly provides building blocks like ADD, ADC, SUB, and SBB.
- Using these, we can extend addition and subtraction to any number of bits.
- ADC (Add with Carry) and SBB (Subtract with Borrow) use the carry flag to link operations.

Extended Addition Algorithm

Concept

- ADD performs normal addition.
- ADC adds with the carry flag — useful for the upper half addition.

Assembly Code

```
dest: dd 40000
```

```
src:  dd 80000
```

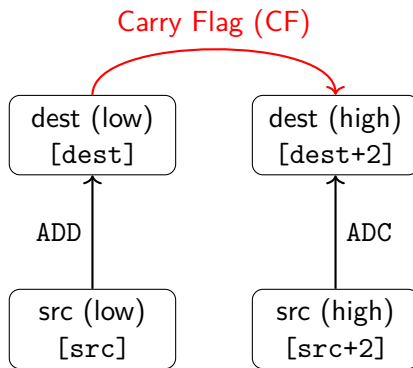
```
mov ax, [src]
```

```
add word [dest], ax
```

```
mov ax, [src+2]
```

```
adc word [dest+2], ax
```

Memory Visualization: Extended Addition



Extended Subtraction Algorithm

Concept

- SUB performs normal subtraction.
- SBB (Subtract with Borrow) subtracts with the carry flag (used as borrow).

Assembly Code

```
dest: dd 40000
```

```
src:  dd 80000
```

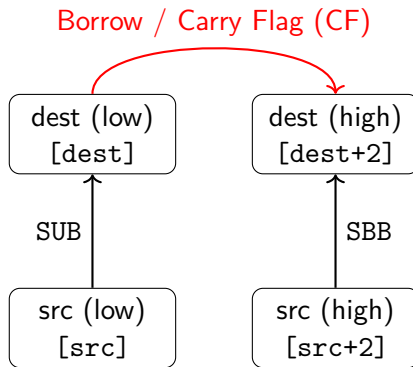
```
mov ax, [src]
```

```
sub word [dest], ax
```

```
mov ax, [src+2]
```

```
sbb word [dest+2], ax
```

Memory Visualization: Extended Subtraction



Key Takeaways

- ADD and SUB handle normal 16-bit operations.
- ADC and SBB extend them across word boundaries.
- Carry flag plays a crucial role in linking multi-word operations.
- Using these techniques, we can perform operations on numbers of any size.

Concept of Extended Multiplication

- Uses **extended shifting** and **extended addition**.
- Multiplier: 16-bit (bits checked one by one).
- Multiplicand: 32-bit (to avoid loss of significant bits during shifts).
- Result: 32-bit (to hold the full product).
- Algorithm performs bit-by-bit multiplication with shift and add.

Example 4.2 — 16-bit Multiplication Code (Part 1)

Assembly Code

```
[org 0x0100]
jmp start

multiplicand: dd 1300      ; 16-bit multiplicand (32-bit space)
multiplier:   dw 500       ; 16-bit multiplier
result:       dd 0         ; 32-bit result

start:  mov cl, 16          ; initialize bit count to 16
        mov dx, [multiplier] ; load multiplier in DX

checkbit: shr dx, 1         ; move rightmost bit into carry
          jnc skip          ; skip addition if bit = 0
```

Example 4.2 — 16-bit Multiplication Code (Part 2)

Assembly Code (continued)

```
mov ax, [multiplicand]
add [result], ax    ; add lower word
mov ax, [multiplicand+2]
adc [result+2], ax  ; add upper word with carry
```

skip:

```
shl word [multiplicand], 1
rcl word [multiplicand+2], 1 ; shift multiplicand left
```

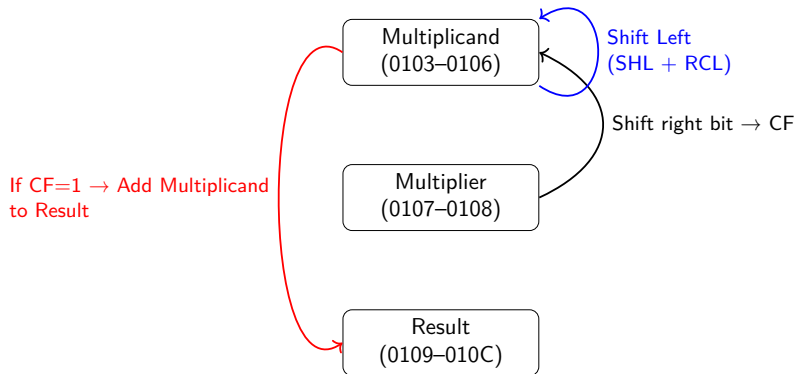
```
dec cl                ; decrement bit count
jnz checkbit          ; repeat if bits left
```

```
mov ax, 0x4c00        ; terminate program
int 0x21
```

Explanation of the Algorithm

- Multiplicand and result occupy 32-bit memory.
- Multiplier stored as a 16-bit word in memory and loaded into DX.
- Each bit of the multiplier is tested by shifting right.
- If the carry flag = 1 $\rightarrow$ multiplicand added to result.
- Multiplicand is shifted left using extended shifting (SHL, RCL).
- Process repeats until all 16 bits of the multiplier are processed.

Memory Layout and Shifting Process



Memory Addresses:

0103-0106 (Multiplicand) | 0107-0108 (Multiplier) | 0109-010C (Result)

Observation in Debugger

- Multiplicand initially: 00000514 (1300 decimal)
- After 16 shifts: 05140000
- Final result: 0009EB10 (65000 decimal)
- Observe memory updates during execution:
 - 0103-0106 → Multiplicand
 - 0107-0108 → Multiplier
 - 0109-010C → Result
- Demonstrates how extended shifting and addition emulate 32-bit operations on a 16-bit system.

Key Takeaways

- Combines extended shifting and addition for full 32-bit multiplication.
- Enables $16\text{-bit} \times 16\text{-bit} \rightarrow 32\text{-bit}$ result using only 16-bit operations.
- Carry flag connects lower and upper word operations seamlessly.
- Demonstrates scalability of basic instructions to larger bit sizes.
- The same principle extends to even larger bit-widths.

Thank You!

Any Questions?

- **Topics Covered:** Shift & Rotate Instructions, Extended Addition, Binary Multiplication.
- **Key Lessons:**
 - **Shift Operations:** Useful for arithmetic manipulations — multiply/divide by 2.
 - **Rotate Operations:** Enable circular bit transfers for cryptography and checksum logic.
 - **Carry Flag:** Essential in rotate-through-carry and extended (multi-byte/word) arithmetic.
- **Takeaway:** Understanding extended shift and rotate logic enables precise **bit-level control** and efficient algorithm design.